ADVANCED OBJECT-ORIENTED DATA STRUCTURES

Heterogeneous Dynamic List

Python-Style List Container Architecture in Modern C++17

A production-grade C++ container implementing dynamic heterogeneity, safe type erasure (std::any), proxy-driven conversions, SFINAE compile-time introspection, and full Pythonic indexing semantics.

PROJECT AUTHORS & CONTRIBUTORS

Akhshay Arulpragasam • Jeevan R

Core Technologies: Modern C++17 | CMake 3.20+ | CTest Framework | Proxy Pattern | Domain Exception Architecture

Problem Statement

The Core Challenge: C++ STL containers enforce strict compile-time type homogeneity (e.g. *std::vector<T>* holds exactly one type). How can we achieve Python's dynamic list flexibility in C++ while preserving strict type safety, zero memory leaks, and high performance?

LIMITATIONS OF C++ STL

Shortcomings of Conventional Approaches

• **Raw void* Pointers:** Unsafe, strips all type information, requires manual casts, vulnerable to memory corruption, zero RAII.

• **Inheritance (Common Base):** Intrusive design requiring all types to derive from a root class. Incompatible with native primitives (int, double, std::string).

• **std::variant:** Strictly closed union: requires all types to be hardcoded at compile time; cannot accept arbitrary new types.

• **Fragile Error Handling:** Silent type mismatches cause undefined behavior or crashes without meaningful exception diagnostics.

• **Absence of Python Semantics:** Zero native support for negative indexing (list[-1]), step slicing, or polymorphic stream printing.

OUR ARCHITECTURE & GOALS

Target Engineering Objectives

✔ **Open-Ended Heterogeneity:** Seamlessly store any C++ type (int, float, std::string, custom structs, nested PythonList) in one container.

✔ **Safe Dynamic Type Erasure:** Leverage std::any with typeid validation and granular domain exceptions (TypeMismatchError).

✔ **Proxy Pattern Ergonomics:** Nested ProxyElement for intuitive element access, transparent implicit conversion, and clean assignment.

✔ **Full Pythonic Semantics:** Support negative indexing (list[-1]), boundary clamping, multidirectional slicing, and std::cout << list printing.

✔ **Production Grade Packaging:** Robust CMake build pipeline with target export, versioning, automated CTest harness, and zero dependencies.

ENVIRONMENT, TOOLING & TECHNOLOGY STACK

Softwares We Used

ISO C++17 CORE

Language & STL Primitives

• **std::any & any_cast:** Core type erasure holding arbitrary copyable objects with full RAII lifetime management.

• **Registry:** Runtime type identification via std::type_index for custom dynamic polymorphic dispatch.

• **SFINAE & std::void_t:** Compile-time introspection detecting IsPrintable and IsEqualityComparable traits.

• **std::reference_wrapper:** Zero-overhead non-owning reference semantics enabled via appendReference().

• **Zero Third-Party Deps:** Built 100% on standard ISO C++17 library for complete cross-platform portability.

MODERN CMAKE

Build & Packaging Pipeline

• **CMake 3.20+ Design:** Modern target-based design with target_include_directories and target_compile_features.

• **GNUInstallDirs Standard:** Compliant installation directory hierarchies for headers, static/shared libs, and binaries.

• **Package Configuration:** Generates PythonListConfig.cmake & Version file for simple find_package() downstream usage.

• **Automated CTest Suite:** Integrated test runner for rapid regression testing and diagnostic reporting.

• **Automation Scripts:** PowerShell & Shell scripts facilitating automated build, test, and install cycles.

TOOLCHAIN & TEST

Toolchains & Verification

• **MSYS2 UCRT64 / MinGW:** Primary Windows development and testing environment leveraging GCC 14+ toolchain.

• **Clang & LLVM Integration:** Validated with Clang 13+ under strict compiler flags (-Wall -Wextra -Wpedantic).

• **MSVC Compatibility:** Clean compilation and execution on Microsoft Visual Studio 2019/2022 conforming modes.

• **Custom Test Harness:** 50+ automated unit & regression assertions covering nested lists, slicing, and memory integrity.

• **Diagramming Tools:** LaTeX TikZ for precise architectural blueprints and PlantUML for structural modeling.

OOP Concepts Involved

CORE ENCAPSULATION

1. Encapsulation & Abstraction

- **Data Encapsulation:** Internal storage is privately managed as std::vector to protect state integrity.
- **Custom Type Registry:** A static std::unordered_map maps std::type_index to custom print callbacks.
- **Clean Abstraction:** Exposes high-level operations (append, insert, pop, slice) hiding complex std::any mechanics.

BEHAVIORAL PATTERN

2. Proxy Design Pattern

- **Lazy Conversion:** operator[] returns a lightweight ProxyElement holding an internal any* reference.
- **Implicit Casts:** operator T() allows seamless assignment into target C++ primitives (e.g. int x = list[0]).
- **Transparent Assignment:** ProxyElement::operator=(const T&) mutates container elements in-place with type checking.

EXCEPTION ARCHITECTURE

3. Inheritance & Domain Exceptions

- **Base Exception:** All custom exceptions inherit from std::runtime_error for clean universal catching.
- **TypeMismatchError:** Thrown when attempting an incompatible any_cast extraction.
- **UnsupportedOperationError:** Thrown on unprintable or non-comparable user-defined types.
- **ArgumentNotPresentError:** Thrown when search operations (index, remove) fail to locate the element.

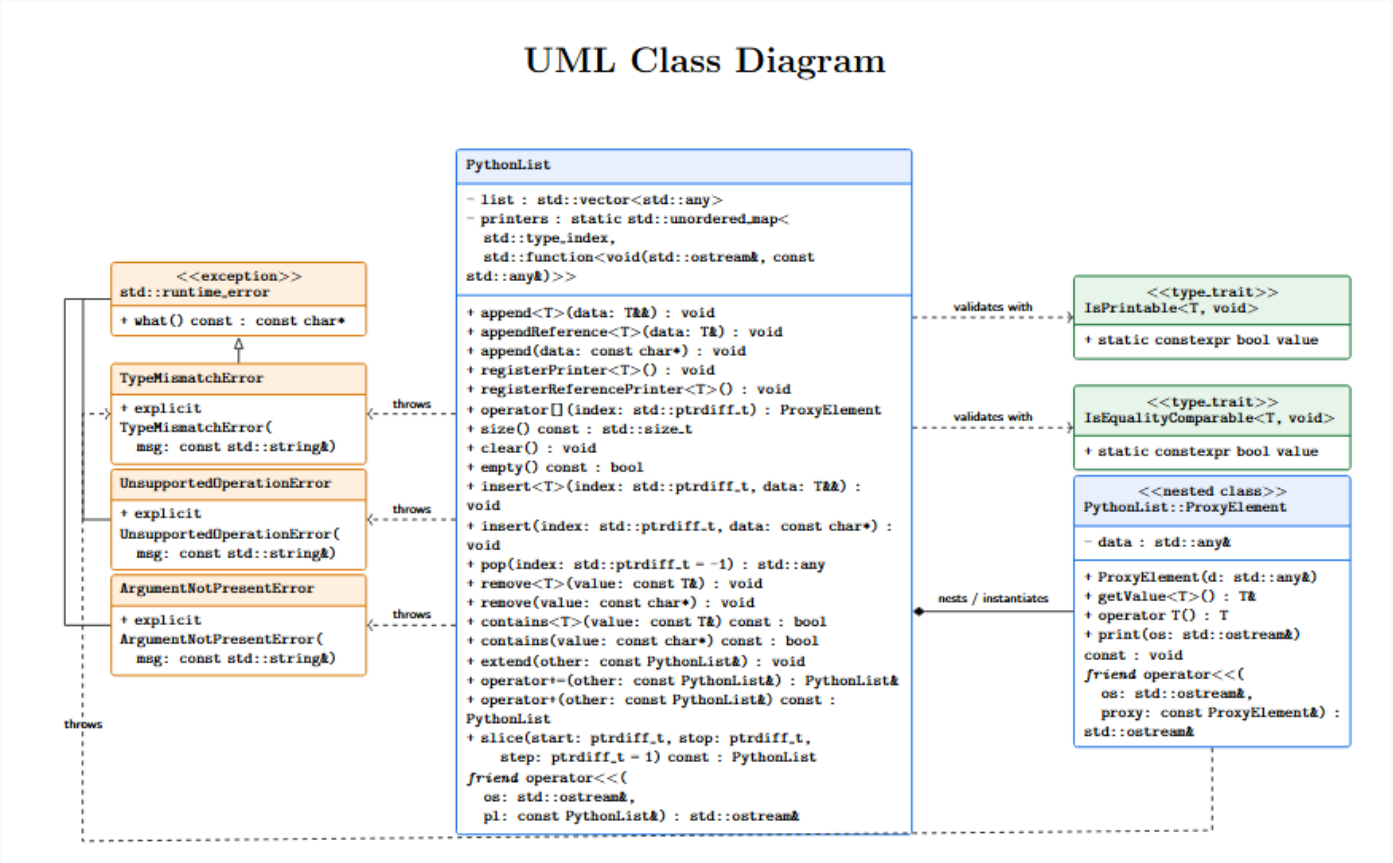
METAPROGRAMMING

4. Polymorphism & Metaprogramming

- **Dynamic Polymorphism:** Dispatch registry formats stream printing per type without requiring a common base class.
- **Compile-Time SFINAE:** Type traits IsPrintable and IsEqualityComparable statically guard template instantiation.
- **Deep Operator Overloading:** Overloaded operator[], operator+, operator+=, and operator<< provide authentic Python ergonomics.

Class UML Diagram

STRUCTURAL UML 2.0 SPECIFICATION



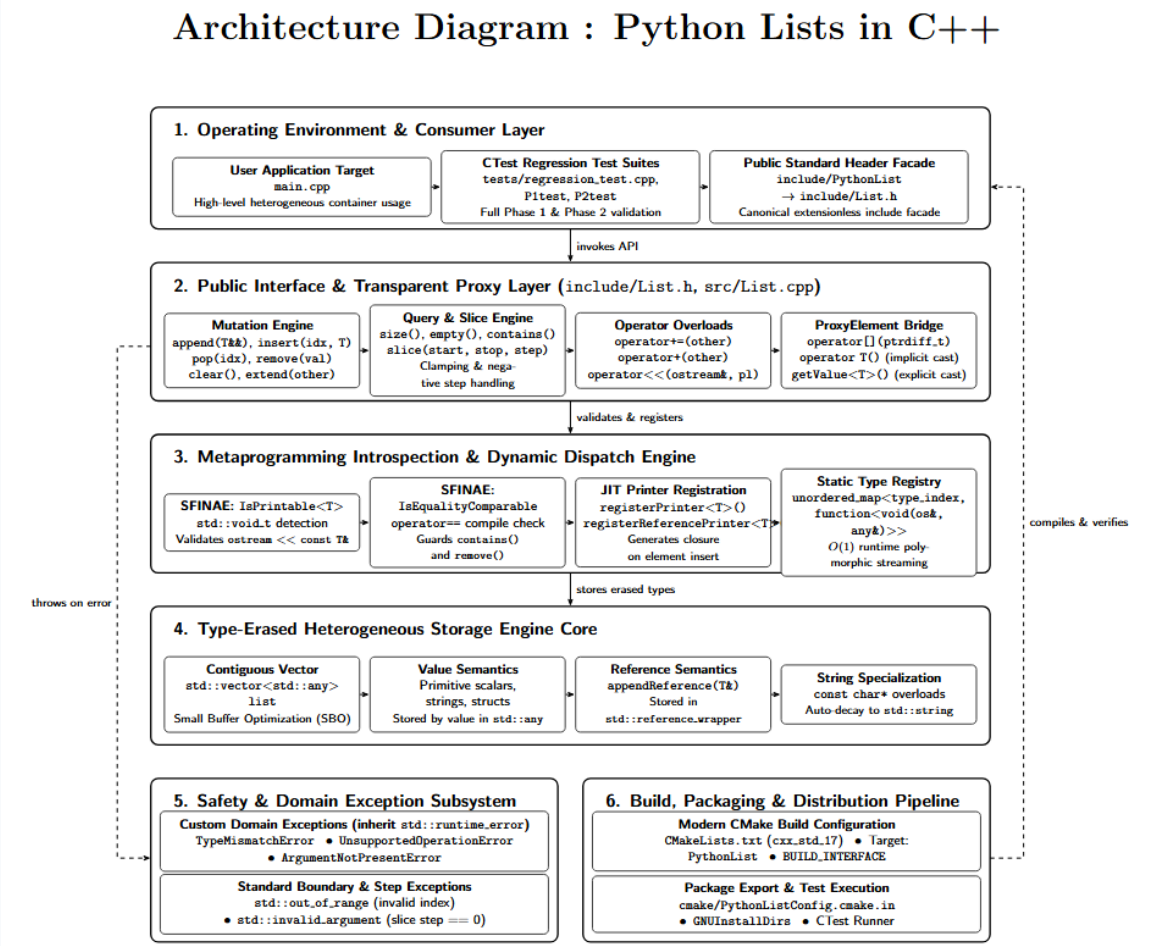
CLASS RELATIONSHIPS

Key UML Entities

- **PythonList (Core Container):** Main class encapsulating std::vector storage, dynamic registry, and public container API.
- **ProxyElement (Nested Proxy):** Composition structure inside PythonList. Holds pointer/index reference to resolve subscript conversions lazily.
- **Exception Subsystem:** Inherits publicly from std::runtime_error. Defines TypeMismatchError, UnsupportedOperationError, etc.
- **Metaprogramming Traits:** SFINAE helper templates (IsPrintable, IsEqualityComparable) guiding conditional compile-time dispatch.
- **Polymorphic Registry:** Static unordered map mapping std::type_index to lambda function pointers for custom printing.

Architecture Diagram

TIERED ARCHITECTURAL BLUEPRINT



SUBSYSTEM DECOMPOSITION

Tiered Subsystem Breakdown

- **1. Consumer Layer:** Application client code, automated test runners, and downstream CMake modules.
- **2. Interface & Proxy Layer:** Public member functions, overloaded operators ([] , +, +=, <<), and ProxyElement conversion bridge.
- **3. Dynamic Dispatch Engine:** Runtime type reflection via typeid / std::type_index and SFINAE introspection rules.
- **4. Storage Core:** Underlying std::vector container handling heap allocation and object lifecycles.
- **5. Safety & Diagnostics:** Domain-specific exception handlers enforcing strict boundary and type safety.
- **6. Build & Packaging Pipeline:** Modern CMake 3.20+ modular target exports and CTest verification suite.

Key Features & Summary

EXPRESSIVE INTERFACE

Pythonic Semantics

• **Negative Indexing:** Direct access from list tail (e.g. list[-1]) matching Python 1-to-1.

• **Multi-Directional Slicing:** Supports slice(start, stop, step) including reverse indexing and step intervals.

• **Boundary Clamping:** Automatically normalizes extreme bounds without memory segmentation faults.

• **Polymorphic Printing:** Formats entire list with std::cout << list matching [1, 3.14, 'hello', [True]].

ROBUST TYPE SAFETY

Memory & Type Safety

• **Full RAII Compliance:** Automatic lifecycle and heap memory reclamation through std::any destructors.

• **Value vs Reference:** Offers copy semantics via append() and live mutable references via appendReference().

• **Strong Exception Safety:** Mutating operations maintain container consistency if exceptions occur.

• **Zero Memory Leaks:** Valgrind and AddressSanitizer verified leak-free operation on deeply nested lists.

PROJECT TAKEAWAYS

Key Takeaways

• **Modern C++ Mastery:** Successfully combined runtime type erasure with compile-time template metaprogramming.

• **Design Patterns in Action:** Applied Proxy, Factory, and Registry patterns cleanly without overhead.

• **Production Packaging:** Engineered ready-to-deploy modern CMake library export for enterprise usage.

• **Seamless Usability:** Delivered genuine Python list ergonomics with pure C++ native speed.